

Testing



Testing Levels

Software can be tested at different levels

1. Unit Tests - test individual components
2. Integration Tests - test groups of components
3. Acceptance Tests - test the client needs

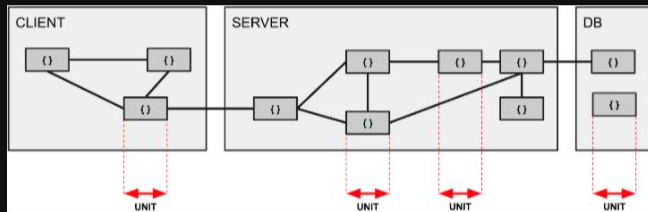


Unit Tests



The smallest testable part of an application, could be an entire class, a single method or a specific section of code. Each test must:

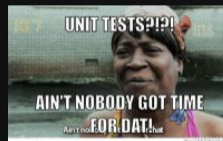
- check one assumption, and one assumption only
- execute quickly
- be independent from other tests
- produce consistent results



The unit to be tested is referred to as System Under Test - SUT

Value of Unit Testing

Most developers do not like to write tests!



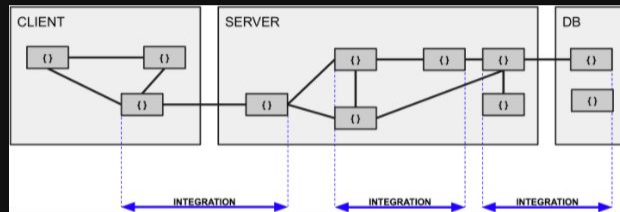
The only certainties in life are death, taxes and bugs in code.

- Unit tests prove that your code actually works
- They provide confidence when refactoring
- You can improve the design without breaking it
- More fun to code with them than without
- They demonstrate concrete progress
- Unit tests are a form of documentation for the code
- Unit tests make better designs
- Faster than writing code without tests!

Integration Tests

Integration tests confirm that two separate units can work together by testing the interfaces between them

Each module should be unit-tested in order to guarantee we are testing their interaction and not their individual behaviour

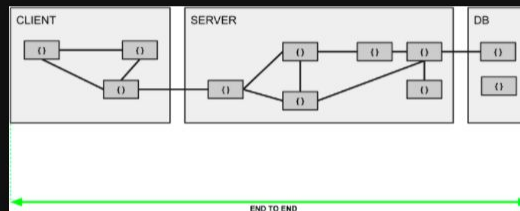


Acceptance Tests

Often called **end-to-end** tests,
they validate the application as a whole

Tightly coupled with client requirements, concerned mostly about
user experience not implementation details

High number of variables present, hard to create and maintain



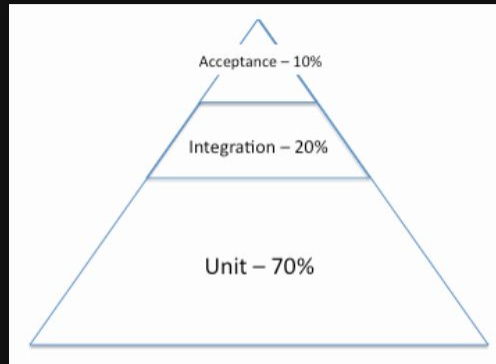
Require some kind of user interaction, automated or not, which makes them a lot slower than unit/integration tests

Test Distribution

Unit tests are independent, easier to develop and fast to execute

They should be fully integrated with the application build process

Mike Cohn created a concept called Test Pyramid that distributes the amount of tests you should do for each scope:



- Unit Tests: 70%
- Integration Tests: 20%
- Acceptance Tests: 10%

Test Structure

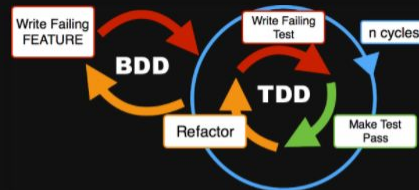
Each test should be separated into four distinct segments:

1. **Setup** - instantiates and establishes the state of the system under test prior to any test-related activities
2. **Exercise** - invoke methods of the system under test, which represent a certain behavior the test intends to verify
3. **Verify** - verify whether the expected outcome of a given behavior has been obtained
4. **Teardown** - cleaning up, making sure test is fully isolated and does not interfere with subsequent tests

Test Driven Development

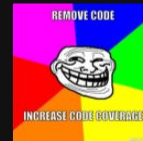
A style of programming software that relies on the repetition of a very short development cycle:

1. Add a test describing an aspect of the program
2. Run the test, which should fail because the program lacks that feature
3. Write the simplest possible code to make the test pass
4. Refactor the code, making sure:
 - the code contains no duplication
 - the code expresses separately each distinct idea or responsibility
 - the code is composed of the minimum number of components
5. Repeat, accumulating tests over time



Test Coverage

Measure used to describe the degree to which the source code of a program is executed when tests are ran



Metrics:

- Class Coverage
- Method Coverage
- Line Coverage
- Branch Coverage

Coverage Report - All Packages

Package /	# Classes	Line Coverage	Branch Coverage	Complexity
All Packages	38	50%	62%	1.078
net.jvw.fdb5	3	77%	91%	1
net.jvw.fdb5.logic	3	57%	73%	0
net.jvw.fdb5.lucene	2	45%	58%	0
net.jvw.fdb5.model	5	66%	80%	1.286
net.jvw.fdb5.model.ibatis	9	70%	N/A	1
net.jvw.fdb5.model.rss	1	69%	0%	1.222
net.jvw.fdb5.struts	13	21%	29%	1.571
net.jvw.taqlibs.tooltip	2	0%	N/A	1.2

Report generated by [Cobertura](#) 1.7 on 3/13/06 12:52 PM.

Testing in Java

JUnit

JUnit

Testing framework to write and run repeatable tests

Provides:

- Assertions for testing expected results, making test pass or fail
- Test Fixtures for sharing common test data, ensuring a well known and fixed environment for tests
- Test Runners for running tests and outputting results

Adding JUnit to your maven project

```
<dependency>  
  <groupId>junit</groupId>  
  <artifactId>junit</artifactId>  
  <version>4.12</version>  
  <scope>test</scope>  
</dependency>
```

Writing a test

Let's do some extreme TDD

Your client needs a calculator that only sums positive numbers. If an operand is negative, a runtime exception is thrown

Before writing the calculator code, let's create some tests for this customer requirement

Sketching the Calculator

```
public class Calculator {  
    public int sum(int a, int b) {  
        return 0;  
    }  
}
```

Writing the tests

```
public class CalculatorTest {  
  
    // the Calculator is our SUT  
    private Calculator calculator;  
  
    // this code will run before each test  
    @Before  
    public void setup() {  
        calculator = new Calculator();  
    }  
  
    // this annotates this method as a test, and will be invoked by the Test Runner  
    @Test  
    public void sumTest() {  
  
        int result = calculator.sum(10, 5);  
        int expected = 15;  
  
        // assert if two values are equal. if they are different, the test fails  
        assertEquals(expected, result);  
    }  
  
    // this test expects an IllegalArgumentException, and will fail if it was not thrown  
    @Test(expected = IllegalArgumentException.class)  
    public void sumNegativeTest() {  
        calculator.sum(-10, 10);  
    }  
}
```

Be assertive

Have you noticed the method used to compare a result against an expected value? That's right, assertEquals()!

This is one of the various assertions that you can make with JUnit:

Method	Description
assertTrue(test)	fails if the Boolean test is false
assertFalse(test)	fails if the Boolean test is true
assertEquals(expected, actual)	fails if the values are not equal
assertSame(expected, actual)	fails if the values are not the same (by ==)
assertNotSame(expected, actual)	fails if the values are the same (by ==)
assertNotNull(value)	fails if the given value is not null
assertNotNull(value)	fails if the given value is null
fail()	causes current test to immediately fail

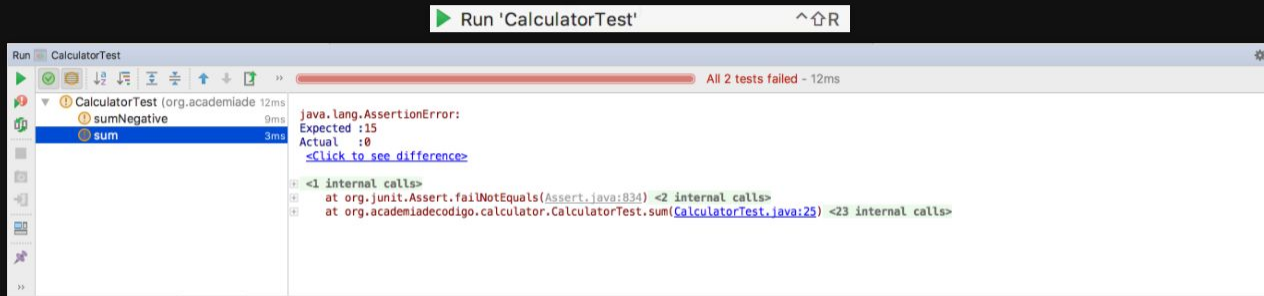
Each method can also receive as a first argument a failure message, clarifying the reason and point of failure.

```
assertEquals("result is different from expected value", expected, actual);
```

Running the test

Although JUnit provides its own console-based **Test Runner**, most IDEs include one with a GUI for manually running tests and analyzing results

In order to run tests in IntelliJ IDEA, right-click a test package, class or method and select Run



IntelliJ IDEA built-in **Test Runner**

Make it work

```
public class Calculator {  
    public int sum(int a, int b) {  
        if (a < 0 || b < 0) {  
            throw new IllegalArgumentException();  
        }  
        return a + b;  
    }  
}
```

Run the tests again and see them pass. Feels good, right? That's TDD for you.

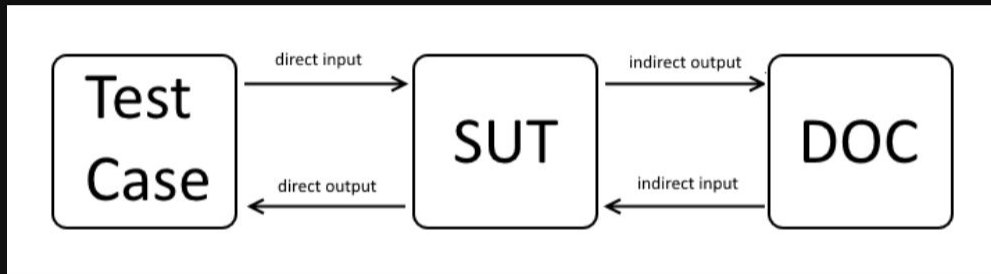
You can now make confident changes to the Calculator code, guaranteeing that new features will not break existing functionality

Test Isolation

Sometimes the system under test depends on other objects or external components, such as a database driver or a file system API, that are not part of the SUT

These are called depended-on components - **DOC**

- DOC failures may affect the correctness of our tests
- DOCs may slow down test execution
- Unexpected changes in DOC behaviour makes tests fragile and unreliable



In short, we want to test our SUT, not our DOCs

Depended-On Components

In our application, controllers depend on services

Services usually query data from a database. Any failure in this connection will affect the controller tests

```
public class LoginController {  
  
    private UserService userService;  
  
    public void doLogin(String username, String password) {  
  
        if (userService.authenticate(username, password)) {  
            // show success page with logged-in user name  
        } else {  
            // show error page  
        }  
    }  
}
```

Stubs and Mocks

To proper SUT isolation, DOCs are replaced by Stubs or Mocks

They are lightweight replacements that we control
and simulate the desired behaviour

```
public class UserServiceMock implements UserService {  
  
    private Map<String, User> users;  
  
    @Override  
    public boolean authenticate(String username, String password) {  
  
        User u = users.get(username);  
  
        if (u == null) {  
            return false;  
        }  
  
        return password.equals(u.getPassword());  
    }  
}
```

Stubs and Mocks

This alternative implementation of the same interface will allow us to isolate LoginController in its tests.

```
public class LoginControllerTest {  
  
    private LoginController loginController;  
  
    public void successLogin() {  
  
        UserService userService = new UserServiceMock();  
        userService.addUser(new User("rui", "ferrao"));  
        loginController.setUserService(userService);  
  
        loginController.doLogin("rui", "ferrao");  
        // check if the success page has been shown  
        // check if it has written "rui" in it  
    }  
}
```

There must be a better way

Creating mocks for every single DOC
of our tests can become cumbersome

Presenting...



Mockito

Tasty mocking framework for unit tests in Java

```
public class LoginControllerTest {  
  
    private LoginController loginController;  
  
    public void successLogin() {  
        UserService userService = Mockito.mock(UserService.class);  
        loginController.setUserService(userService);  
  
        // if authenticate method is called with "rui" and "ferrao", the mock returns true  
        Mockito.when(userService.authenticate("rui", "ferrao")).thenReturn(true);  
  
        loginController.doLogin("rui", "ferrao");  
  
        // verify that authenticate method has been called with the correct arguments  
        Mockito.verify(userService).authenticate("rui", "ferrao");  
  
        // check if the success page has been shown  
        // check if it has written "rui" in it  
    }  
}
```

Component Interactions

Although we're able to define each method's return value, Mockito's main focus is the interaction between objects

Noticed how we verified a method invocation on the mock?

Mockito allows us to verify these interactions, which will also make the test pass or fail

Interaction Verification

Mockito's API allows verification of method calls with great detail

```
import static org.mockito.Mockito.*; // import all static methods to this class

Glass glass = mock(Glass.class); // create a mock
bar.makeMojito(glass); // test a bar

// verify that the add method was called with "Rum" as argument
verify(glass).add("Rum");
verify(glass).add("Sugar"); // same for sugar

// verify that add method was called with Lime and a float as arguments
verify(glass).add("Lime", anyFloat());

// verify that addIce method was called three times
verify(glass, times(3)).addIce();
verify(glass).add("Água das Pedras");

// verify that add was never called with "Water" as argument
verify(glass, never()).add("Water");
```

Absolute Control

When your System Under Test depends on data from its DOCs,
interaction verification is not enough

```
import static org.mockito.Mockito.*;

Client client = mock(Client.class);

// return 18 when getAge() is called
when(client.getAge()).thenReturn(18);

// throw an exception when pay is called with an integer
when(client.pay(anyInt())).thenThrow(new RuntimeException());
```

Mockito's API allows for easy and flexible configuration of method call return values

Adding Mockito to your maven project

```
<dependency>  
  <groupId>org.mockito</groupId>  
  <artifactId>mockito-core</artifactId>  
  <version>2.21.0</version>  
  <scope>test</scope>  
</dependency>
```

*Always code as if the guy who ends up maintaining
your code will be a violent psychopath who knows
where you live* [Martin Golding]

{ Exercise }

Unit test JavaBank

